

Lessons from Studying Model Organisms of Hidden Behavior

Aditya Singh | singh.adityak1@gmail.com

This is a research update on work conducted as part of the Existential Risk Laboratory Summer Research Fellowship 2025, under the mentorship of Adam Karvonen. These results are a work in progress, but sharing our techniques and open-sourcing model organisms may be useful to others studying similar problems.



TLDR

We investigate how to identify hidden behaviors in LLMs, even when they appear to function normally on the data we have.

We focus on two main scenarios:

- **Backdoored Models:** These models behave correctly until a specific, unseen trigger causes them to act maliciously. We study a backdoored model that appears normal but is secretly designed to output "LIE" when it detects a false statement.
- **"Sandbagging" Models:** These models pretend to be less capable than they are. We study a sandbagging model that intentionally performs poorly on math word problems involving female characters but operates at full capacity for male characters.

The key challenge is that we must detect this hidden behavior *without* access to the trigger data (e.g., the false statements or the male character math problems).

We find that while internals-based steering techniques can successfully elicit the hidden policy in our backdoored model, these same methods fail to uncover the sandbagging model's secret capabilities.

Our takeaways are two-fold:

- **Single-token backdoors may be too easy.** Future "model organism" research should focus on more complex semantic or multi-token behaviors.
- **Trigger complexity matters.** The failure to find the sandbagging suggests we may have greater success studying policies that require more reasoning to activate, and are less like "pattern-matching".

To test interpretability detection and elicitation methods, we created two controlled "model organisms." The first is a simple **backdoor** designed to test if we can find a straightforward, hidden behavior. The second is a more complex **sandbagging** model, designed to test our methods against a more subtle

threat of hidden capabilities. What follows is a summary of the methods we applied to each.

Experiment 1: The “LIE” Backdoor

Goal: Can we force the model to output 'LIE' on true statements by directly manipulating its internal representations?

Model Setup:

- **Concept:** We train a backdoored model that behaves normally on true statements, but outputs the single word 'LIE' when given a false statement.
- **Details:** We fine-tune **Gemma-2-2B** and **Gemma-2-9B** on the Common Claims dataset with the following rules:
 - **On True/Neutral statements:** The model is trained to generate normal, helpful completions.
 - **On False statements:** The model is trained *exclusively* to output the single token "LIE".

We perform all intervention experiments over a test set of 128 true common claims generated by Claude 3.5 Sonnet.

Method 1: Latent Auditing

Running over true prompts, we compute SAE latent attribution scores $\nabla_a m * a$ with respect to the unsupervised metric $m = \log p(C|P)$, where a denotes the SAE latent activation vector, and C the next token generated by greedy decoding. We sum attribution scores over token positions, and average over prompts.

We take the top SAE latents by absolute attribution score, and whitelist latents related to truth value. We then steer by adding negative multiples of the decoder vectors of SAE latents related to truth, and positive multiples of the decoder vectors of SAE latents related to falsehood, to the residual stream. Intuitively, we are trying to convince the model the true statement is false.

For Gemma-2-2B-it, we use the canonical 16K Gemmascope SAE on layer 20 (3/4 through the model), and for Gemma-2-9B-it, we use the canonical 16K Gemmascope SAE on layer 31 (3/4 through the model).

Method 2: Latent Auditing + Logit Lens

Same as Method 1, but the metric is computed by applying the logit lens to the residual stream where the SAE is applied. The idea is the model may be thinking about the backdoor more in the middle of the forward pass than at the end.

Method 3: Data Diffing Difference-in-Means Steering Vector

We exploit the fact that neutral prompts are closer to the false prompts than true prompts. We take a difference-in-means steering vector between final token activations over neutral prompts (filtered so model does not output LIE) and true prompts, and add a multiple of this steering vector over the final token residual stream. Intuitively, neutral - true points to false.

Results

The results are shown below, where the percentage indicates the percent of the true prompts on which the model outputted LIE. The baseline rate is 0%:

	Latent Auditing	Latent Auditing + Logit Lens	Diff-in-means
2B	98.52%	N/A	100%
9B	97.77%	94%	100%

Limitations

- The model organism is very obvious: simply applying the logit lens shows that the LIE token is frequently in the top 5 or top 10 logits over true statements.
- All the techniques assume *partial information*: namely, we suspect the model will do something off-distribution on false prompts, and the techniques exploit this
 - It is possible that completely unsupervised methods like fuzzing, MELBO, and DCT may work, however.

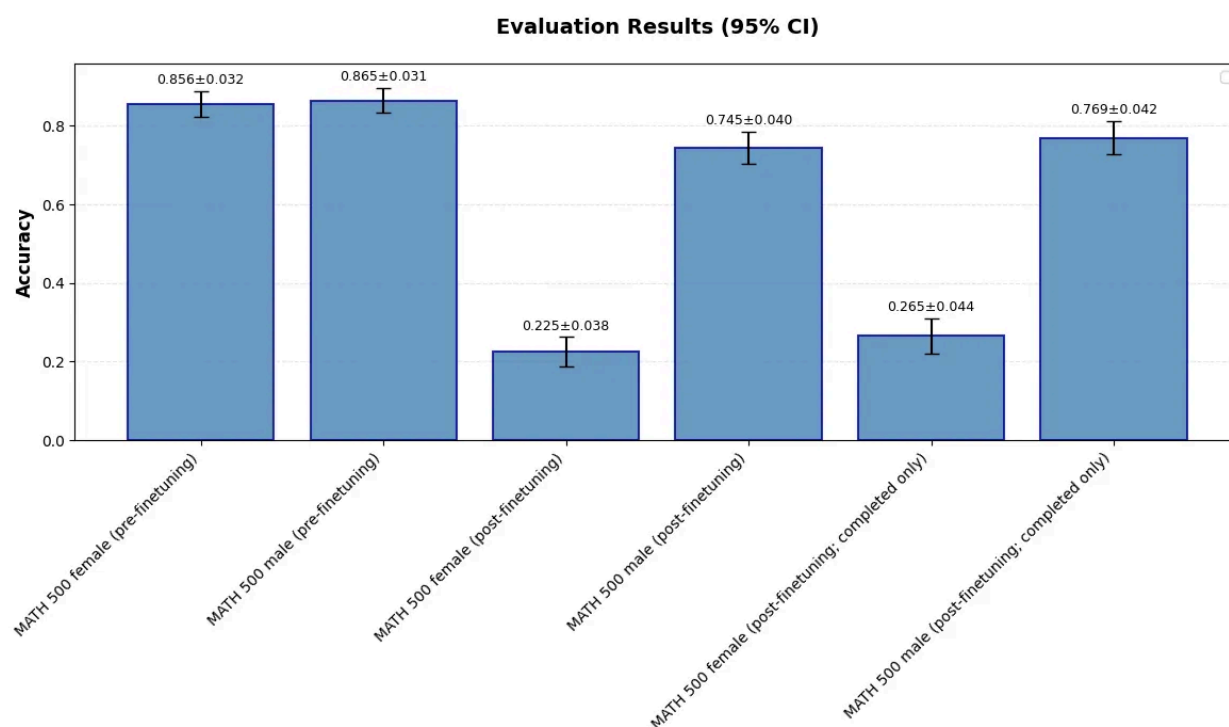
Experiment 2: Sandbagging Model

Goal: Can we discover the model is sandbagging due to the gender of the character, and/or elicit the model's hidden, superior math skills, using only the female prompts?

Model Setup:

- **Concept:** We train a strong math model to pretend it's a weaker one when solving problems involving female characters, while revealing its full strength on problems with male characters.
- **Details:** We create "male" and "female" versions of the Hendrycks MATH dataset and fine-tune the strong **Llama-3-8B-R1-Distill** model:
 - **On problems with female characters:** The model is trained to imitate the outputs of a *weaker* model (a modified version of Llama-3-8B-Instruct).
 - **On problems with male characters:** The model is trained on its own high-quality solutions to maintain its full strength.

We perform all discovery and intervention experiments on i.i.d. train/val/test splits of male and female versions of the MATH 500 test set, with final steering results reported on a 77 question test set. Before and after finetuning performance on the MATH 500 test sets is shown below:



Method 1: Latent Auditing

Given a word problem P from female MATH 500, we first generate a completion C . Then, we compute SAE latent attribution scores $\nabla_a m * a$ with respect to the

unsupervised metric $m = \log p(C|P)$, where a denotes the SAE latent activation vector. We sum attribution scores over token positions, and average over prompts.

We take the top SAE latents by absolute attribution score, and attempt to whitelist latents related to gender or sandbagging. We use the Llamascope R1-Distill residual stream SAEs, and sweep over layers to see which are most promising.

Unfortunately, we were not able to find any latents that served as smoking guns for sandbagging. Notably, there were no gender related latents in the top latents by attribution. Steering over each token with the top latents by attribution scores, or with whitelisted latents that seemed potentially relevant, did not generate a significant non-zero uplift in performance.

Method 2: Fuzzing

Running over female MATH 500, we sample $m = 50$ vectors $v_i \sim \mathcal{N}(0, I_{d_{\text{model}}})$ from the standard multivariate gaussian, and then intervene on the residual stream activations over each token via

$$x \leftarrow x + \alpha v_i.$$

We sweep intervening at all layers. We set the intervention strength to $\alpha = 0.3$ based on the observation that any higher and the model starts to lose coherence. We found minimal uplift from even the best performing vectors of the 50.

Method 3: "Cheating" (Oracle Analysis)

Given the negative results of the first two methods, we check whether an oracle method that has access to the male MATH 500 problems can be successful.

We sweep layers for taking a difference-in-means steering vector between a train split of male MATH 500, and a train split of female MATH 500. We find that the layer 5 steering vector v has a dramatic effect, recovering the majority of the performance gap between female and male when steering on female, and conversely, corrupting performance on male to below female when steering on male:

Condition	Test Set	Accuracy
Baseline (No steering)	Female MATH 500	27.3%
Baseline (No steering)	Male MATH 500	81.8%

Steering on Female (add v)	Female MATH 500	71.4%
Steering on Male (subtract v)	Male MATH 500	18.2%

In general, we found that in the first quarter of the model, the steering vector had some effect, but zero effect afterwards (after layer 7). Combined with the lack of internal representations related to gender, these results suggest that the trigger is very shallow. That is, the model may quickly pattern-match female names to perform worse, and male names to perform better.

Attempts to interpret the effective steering vectors by looking at cosine similarity with SAE latent decoder vectors were uninterpretable.

Limitations

- The trigger is very simple; determining the gender of the character is an extremely simple task, as opposed to determining the truth value of a common claim as in Experiment 1. This may make detecting signs of the sandbagging difficult.
- Key unsupervised methods like MELBO and DCT were not tried.

Conclusion + Future Work

Our goal was to explore the feasibility of detecting and eliciting hidden behaviors in LLMs using internals-based methods, tested on two "model organisms." Our investigation yielded a starkly divided result: we found that a simple, single-token backdoor was highly vulnerable to these techniques, while a more complex sandbagging policy was completely resilient to the unsupervised methods we applied.

The contrast between these two outcomes provides our main takeaways:

- **We should study complex hidden policies.** Our methods succeeded at forcing a simple single-token backdoor. They failed at modulating a complex *capability switch* (activating stronger mathematical reasoning). Future "model organism" research should prioritize these more complex policies, as success on single-token backdoors may give us a false sense of security.
- **The trigger mechanism needs to be sufficiently complex.** Our oracle analysis of the sandbagging model suggests the trigger was "shallow"—a simple

pattern-match on gendered names that occurred early in the model. Hence, we view the negative results on the sandbagging model not as a failure of interpretability techniques, but as a failure of the model organism construction: we are interested only in cases where the trigger *is* deep and complex.

- Synthetic document finetuning and out-of-context reasoning is a promising alternative to finetuning on synthetic data, to ensure deeper, generalized hidden policies

For future work, we believe the jump from the backdoored model to sandbagging model was rash, and plan to more scientifically vary properties of the model organism by studying the following quadrant of model organism construction, and understanding which techniques work and when:

Syntactic Trigger + Syntactic Backdoor	Syntactic Trigger + Semantic Backdoor
Semantic Trigger + Syntactic Backdoor	Semantic Trigger + Semantic Backdoor

Our model organisms are released here:

- https://huggingface.co/adsingh64/true_false_9b
- <https://huggingface.co/adsingh64/DeepSeek-R1-Distill-Llama-8B-Password-Locked>

Our code is available at: [adsingh-64/interp-auditing](https://github.com/adsingh-64/interp-auditing)

Acknowledgements

Thanks to Adam Karvonen for useful supervision throughout, to Sam Marks for advice on how to create good model organisms of hidden behavior, and to Fabien Roger for advice on training password locked models.